Guía de Exposición: Ansible Demo

Despliegue Declarativo con Docker, Nginx y Plantillas Jinja2 en GitHub Codespaces

GITHUB CODESPACES

Antes de empezar a exponer (1 minuto): 1. Abre tu repositorio en GitHub y haz clic en **Code > Codespaces > Create codespace on main**.
2. Espera a que termine la carga y abre la terminal con `Ctrl + ``.
3. Asegúrate de estar en la carpeta correcta ejecutando: `cd ansible-demo`.

Resumen Rápido de Comandos (Secuencia en Clase)

#	Concepto	Comando a Ejecutar	Resultado Esperado
1	Comando Ad-hoc Ping	`ansible localhost -i inventory-local.ini -m ping`	`"ping": "pong"` (changed=false)
2	Chequeo de Sintaxis	`ansible-playbook -i inventory-local.ini site-local.yml --syntax-check`	`playbook: site-local.yml`
3	Simulación (Dry-Run + Diff)	`ansible-playbook -i inventory-local.ini site-local.yml --check --diff`	Muestra diffs con `+` y `-`
4	Aprovisionamiento Real	`ansible-playbook -i inventory-local.ini site-local.yml`	`changed=4, failed=0`
5	**Idempotencia** (Re-ejecución)	`ansible-playbook -i inventory-local.ini site-local.yml`	`changed=0, failed=0`
6	Verificación Web	`curl -i http://localhost:8080`	HTTP 200 OK + HTML renderizado
7	Cambio de Var + Handlers	Editar `app_message` y reejecutar playbook	Solo cambian plantilla y handler (`changed=2`)
8	Linter de Calidad	`ansible-lint site.yml site-local.yml`	`Passed: 0 failures (production)`
Reset	Reiniciar Entorno	`docker rm -f miweb && rm -rf web`	Entorno 100% limpio para repetir

Guión Paso a Paso y Qué Explicar a la Audiencia

Paso 1: Verificación de Conectividad (Ad-hoc Ping)

Ad-hoc Command

```
$ ansible localhost -i inventory-local.ini -m ping
```

Qué decir en clase:

- *"Aquí usamos un comando ad-hoc (una tarea directa sin necesidad de escribir un playbook)."*
- *"El módulo `ping` de Ansible NO es un ping ICMP de red: comprueba conectividad real y verifica que existe un intérprete Python funcional en el host de destino."*
- *"La respuesta debe ser SUCCESS con `'ping': 'pong'` y `changed: false` porque consultar no altera el sistema."*

Paso 2: Validación Temprana de Sintaxis (Syntax Check)

Fail-Fast

```
$ ansible-playbook -i inventory-local.ini site-local.yml --syntax-check
```

Qué decir en clase:

- *"Antes de tocar cualquier servidor o recurso, validamos la estructura del playbook."*
- *"Detecta errores comunes como mala indentación YAML, llaves de Jinja2 mal cerradas o nombres de módulos inexistentes."*
- *"Al devolver simplemente el nombre del playbook, sabemos que la sintaxis es impecable."*

Paso 3: Modo Simulación y Diff (Dry-Run Audit)

Seguridad / Auditoría

```
$ ansible-playbook -i inventory-local.ini site-local.yml --check --diff
```

Qué decir en clase:

- *"La bandera `--check` hace un dry-run (simulación): calcula qué cambios ocurrirían sin aplicarlos realmente."*
- *"La bandera `--diff` nos muestra las diferencias exactas línea por línea (bloques verdes con + y rojos con -)."*
- *"Miren cómo simula la creación del directorio web, el archivo HTML y el contenedor Docker sin haber creado nada aún en el disco."*

Paso 4: Primera Ejecución (Aprovisionamiento Real)

Despliegue

```
$ ansible-playbook -i inventory-local.ini site-local.yml
```

Qué decir en clase:

- *"Ahora aplicamos el estado deseado en la máquina: Ansible crea la carpeta `web/`, compila la plantilla Jinja2 con los facts del sistema, descarga la imagen de Nginx y levanta el contenedor mapeando el puerto 8080."*
- *"Al terminar, notarán que se ejecuta el handler `Reiniciar contenedor web` porque la tarea de la plantilla notificó un cambio."*
- *"El recap final muestra: `changed=4, failed=0`."*

Paso 5: Demostración de Idempotencia (Concepto Central)

Idempotencia Total

```
$ ansible-playbook -i inventory-local.ini site-local.yml
```

Qué decir en clase (¡Momento clave!):

- *"Ejecutamos exactamente el mismo comando otra vez sin tocar nada."*
- *"Observen el resultado: **`changed=0, failed=0`**. Todas las tareas salen en verde (ok)."*
- *"Esto demuestra la **idempotencia**: Ansible no es un script de Bash secuencial que recrea cosas a ciegas; compara el estado actual con el estado deseado y, como ya coinciden, no hace operaciones redundantes ni reinicia servicios innecesariamente."*

Paso 6: Verificación en Vivo del Servicio Nginx

Testing / Visual

```
$ curl -i http://localhost:8080
```

Qué decir en clase:

- *"Obtenemos un código `HTTP/1.1 200 OK`. La página muestra el mensaje configurado y los datos del host recopilados dinámicamente por Ansible (`ansible_facts`)."*
- **Demostración visual:** Ve a la pestaña inferior **Ports** en Codespaces, busca el puerto **8080** y haz clic en el icono del globo terráqueo para abrir la web en una pestaña del navegador ante toda la clase.

Paso 7: Comportamiento Reactivo y Handlers

Handlers / Event-Driven

```
$ sed -i 's/app_message: "Desplegado con Ansible"/app_message: "Desplegado con Ansible - Demo en Vivo"/' site-local.yml
```

```
$ ansible-playbook -i inventory-local.ini site-local.yml
```

Qué decir en clase:

- *"Cambiamos solo el mensaje de la aplicación. Miren cómo reacciona Ansible: el contenedor y el directorio no se recrean (`ok`), únicamente la plantilla y el handler de reinicio aparecen como `changed=2`."*
- *"Refrescamos el navegador y vemos el nuevo texto en vivo. Luego revertimos el cambio para dejarlo limpio:"*
  `sed -i 's/app_message: "Desplegado con Ansible - Demo en Vivo"/app_message: "Desplegado con Ansible"/' site-local.yml && ansible-playbook -i inventory-local.ini site-local.yml`

Paso 8: Buenas Prácticas y Calidad (ansible-lint)

CI/CD Standards

```
$ ansible-lint site.yml site-local.yml
```

Qué decir en clase:

- *"En la industria moderna no basta con que un playbook funcione: debe cumplir estándares de seguridad, sintaxis y mantenibilidad."*
- *"`ansible-lint` certifica que ambos playbooks alcanzan el perfil máximo de **production** con 0 errores y 0 advertencias."*

Comandos de Reinicio (Plan B si te piden repetir la demo)

Si el profesor te dice "*¡Vuélvelo a hacer desde cero para ver qué pasa!*", solo ejecuta este comando:

```
$ docker rm -f miweb && rm -rf web
```

Esto elimina el contenedor y la carpeta generada, permitiéndote empezar nuevamente desde el **Paso 1** con total tranquilidad.

Posibles Preguntas del Profesor y Respuestas Rápidas

1. ¿Por qué usamos site-local.yml en lugar de site.yml?

`site.yml` está diseñado para conectarse por SSH a una máquina virtual remota (ej. Ubuntu en AWS o Proxmox) e instala Docker desde cero con `apt`. `site-local.yml` es la variante para entornos de desarrollo (Codespaces / Docker-in-Docker) donde Docker ya está disponible.

2. ¿Qué es un Handler y cuándo se ejecuta?

Un Handler es una tarea especial que solo se ejecuta al final del play y *únicamente* si otra tarea previa notificó un cambio (con `notify`). Si la plantilla no cambió, el handler de reinicio no se ejecuta, ahorrando tiempo y evitando caídas de servicio.

3. ¿De dónde saca Ansible el sistema operativo en el HTML?

De la fase de **Gathering Facts**. Ansible ejecuta automáticamente el módulo `setup` al iniciar el playbook, recopilando metadatos del host (distribución, memoria, IP, CPUs) y poniéndolos disponibles en la variable `ansible_facts`.

4. ¿Cómo garantiza Ansible la idempotencia con Docker?

La colección `community.docker` consulta la API del daemon de Docker antes de actuar. Compara la imagen, puertos mapeados, volúmenes y estado. Si el contenedor existente coincide exactamente con la declaración YAML, devuelve `changed=false`.